Active Learning with Gaussian Processes: A Bayesian Approach to Learning Functions

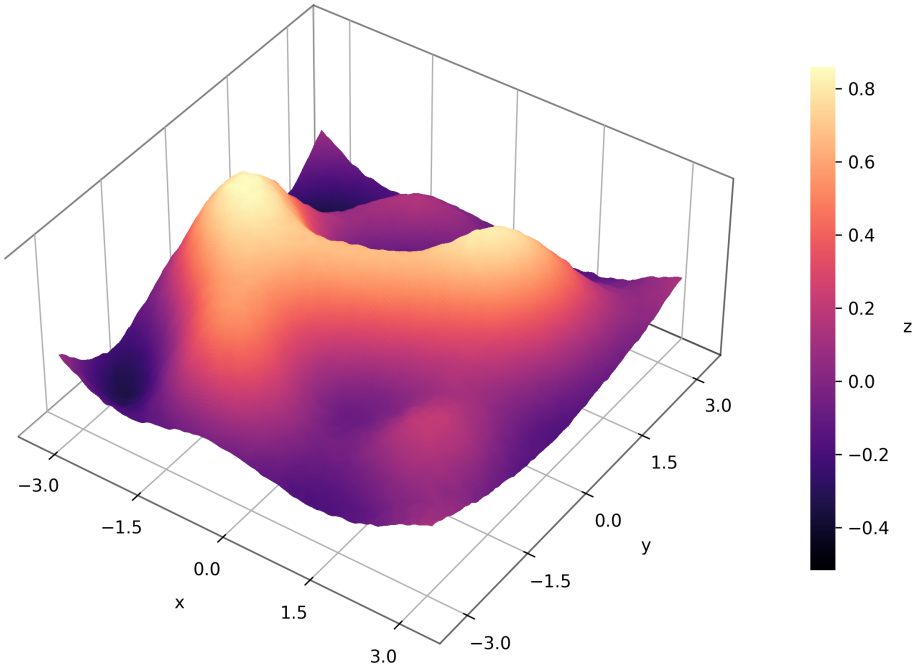
Gavin Kerr

Dalhousie University

March 20, 2026

The Goal of Active Learning

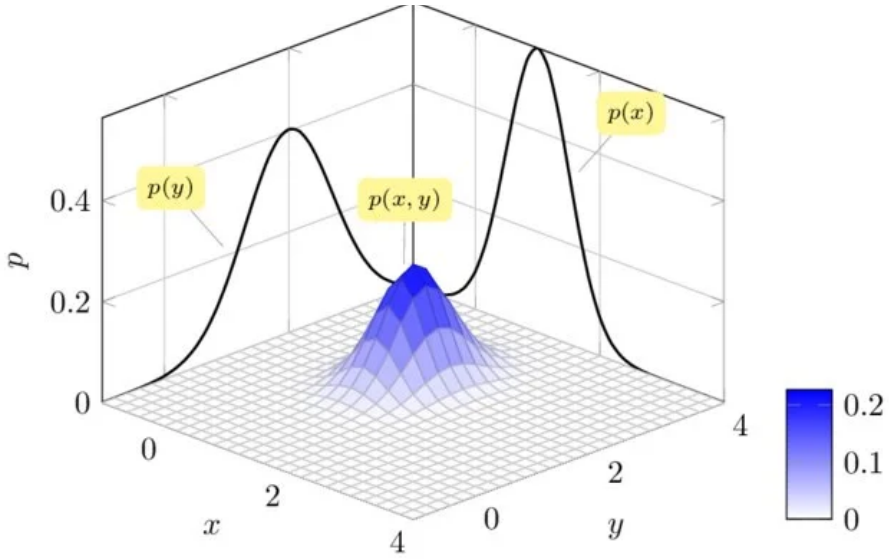
- **Goal:** Learn some unknown function $z(x, y)$ over an input space.
- **Problem:** It's costly to evaluate z.
- **Given:** Prior dataset of N observations of z at points $\{(x_i, y_i)\}_{i=1}^{N}$.
- How to select (x_{N+1}, y_{N+1}) to best improve your model of z?



A Gaussian Process is a Distribution over Functions

- A **multivariate Gaussian**: distribution over vectors.

- Depends on: the mean $\boldsymbol{\mu}$ vector and covariance matrix $\boldsymbol{\Sigma}$.



- A **Gaussian Process** ($\mathcal{GP}$): distribution over functions.

$$f(x) \sim \mathcal{GP}(m(x), k(x, x')).$$

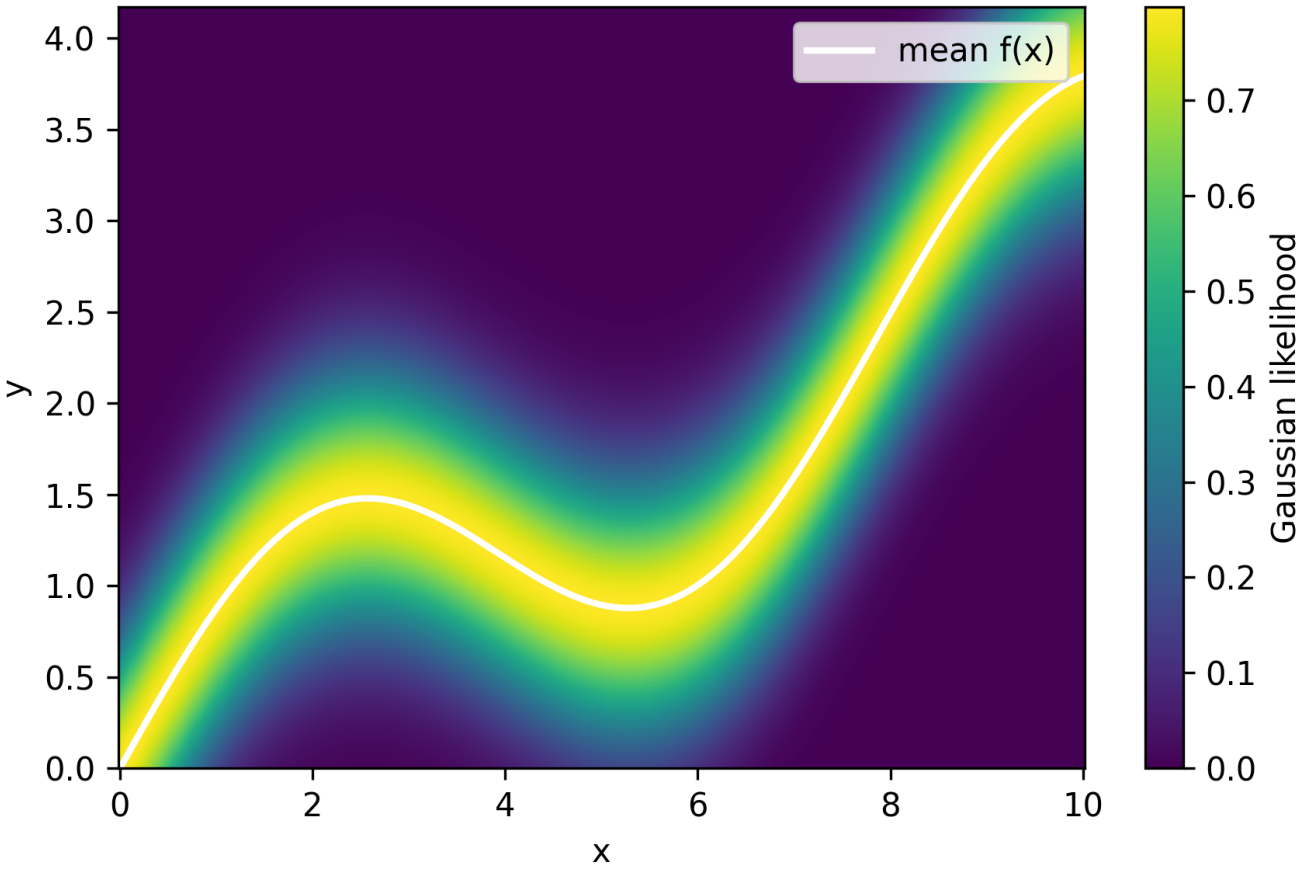
- Depends on: the mean function $m(x)$ and covariance function $k(x, x')$.

[0] Image from: https://stackoverflow.com/questions/61291847/visualizing-a-multivariate-normal-distribution-in-3d-with-python

Gaussian Process Example

$y(x) \sim \mathcal{GP}\big(m(x), k(x, x')\big)$ gives us the **mean** and **uncertainty**.

Key Idea: Sampling $(y(x_1), y(x_2), \ldots)$ gives a multivariate Gaussian.



Bayesian Linear Regression

Model assumptions:

Linear model: $f(\mathbf{x}) = \sum_{j=1}^{d} w_j x_j$.

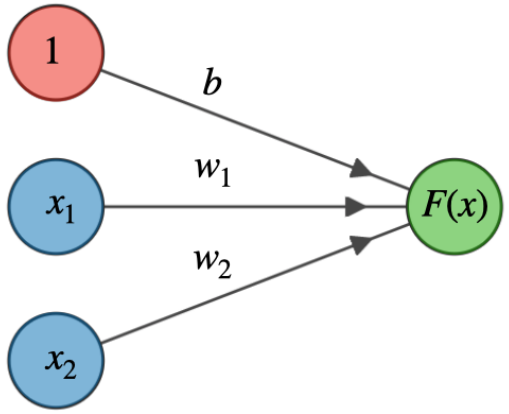
Gaussian prior: $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \sigma_w^2 I)$.

Gaussian noise: $y = f(\mathbf{x}) + \varepsilon$.

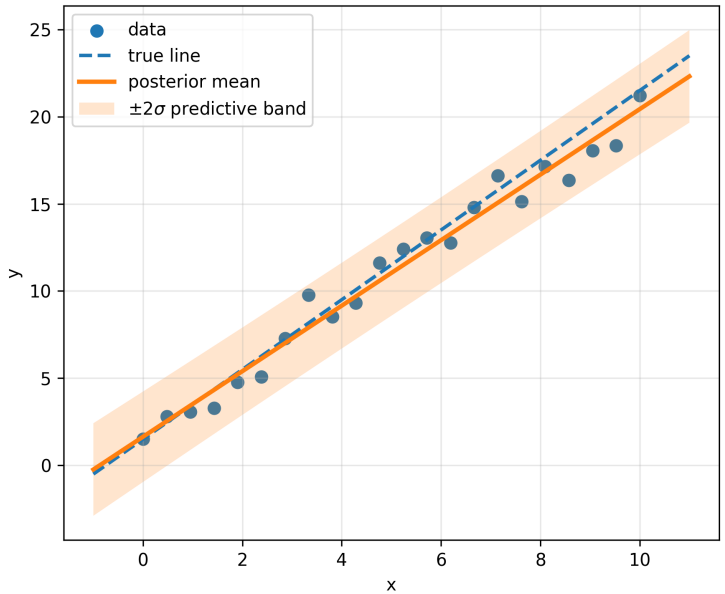
Bayes' theorem $\implies$ the posterior over weights is Gaussian:

$$p(\mathbf{w} \mid X, \mathbf{y}) = \frac{p(\mathbf{y} \mid X, \mathbf{w})\, p(\mathbf{w})}{p(\mathbf{y} \mid X)}.$$

Can use $p(\mathbf{w} \mid X, \mathbf{y})$ to make predictions at new inputs $\mathbf{x}_*$ but there is a better way!



https://joshuagoings.com/2020/05/05/neural-network/



From Linear Models to Infinite Feature Spaces

Problem: Linear models are too simple.

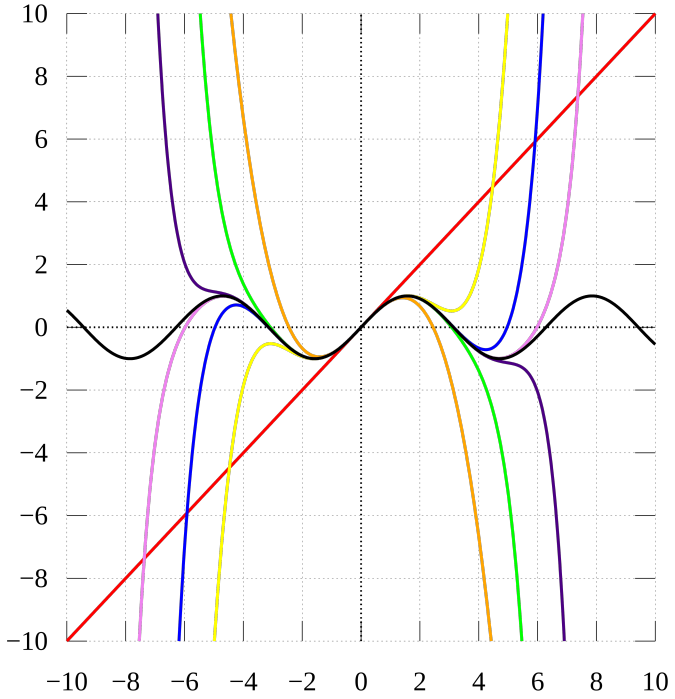
Solution: introduce a feature map:

$$f(\mathbf{x}) = \boldsymbol{\phi}(\mathbf{x})^\top \mathbf{w}.$$

Example: polynomial feature map

$$\boldsymbol{\phi}(\mathbf{x}) = (1, x, x^2, \ldots)^\top \in \mathbb{R}^D.$$

$D \to \infty$ to capture all possible functions.



New Problem: If we want to model all functions we need an infinite number of parameters (weights).

Infinite Features via the Kernel Trick

- Guassian process defined by:

$$m(x) = \mathbb{E}[f(x)], \qquad k(x, x') = \mathbb{E}[f(x)f(x')].$$

- For a linear model with features $\boldsymbol{\phi}(x)$ and $\mathbf{w} \sim \mathcal{N}(0, \sigma_w^2 I)$:

$$m(x) = 0, \qquad k(x, x') = \sigma_w^2 \, \boldsymbol{\phi}(x)^\top \boldsymbol{\phi}(x').$$

- Example feature map:

$$\boldsymbol{\phi}(x) = \left(1, x, \tfrac{x^2}{\sqrt{2!}}, \tfrac{x^3}{\sqrt{3!}}, \dots\right)$$

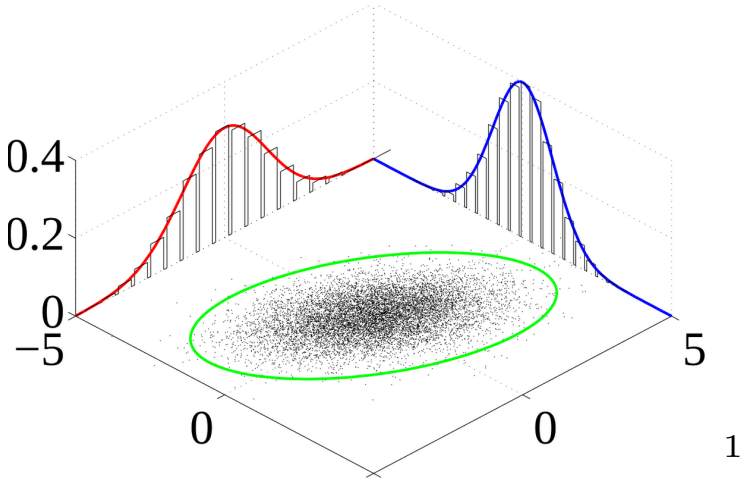
- Inner product becomes:

$$k(x, x') = \sum_{n=0}^{\infty} \frac{(xx')^n}{n!} = e^{xx'}.$$

- **Key idea:** compute kernels directly $\Rightarrow$ implicit infinite features.

Gaussian Process Regression

Sampling a $\mathcal{GP}$ at X and a new point $\mathbf{x}_*$ gives a **joint Gaussian**:



[1]

Red: $\mathbf{y} \sim \mathcal{N}(\mathbf{0}, k(\mathbf{x}_i, \mathbf{x}_j))$ – Prior distribution over observed data.

Blue: $f(\mathbf{x}_*) \sim \mathcal{N}(0, k(\mathbf{x}_*, \mathbf{x}_*))$ – Prior distribution over new point.

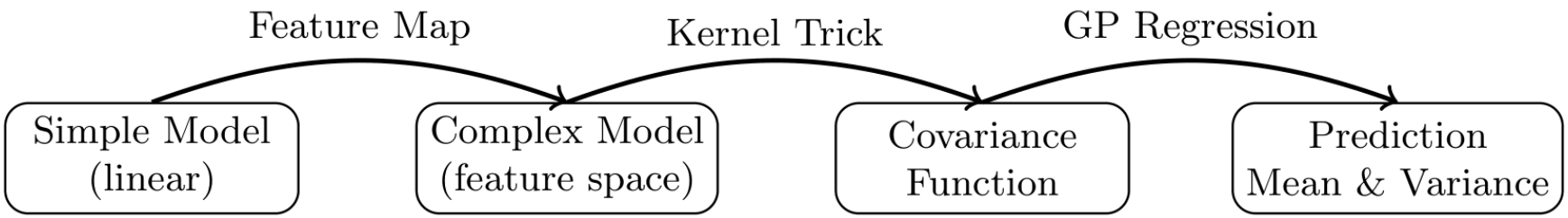
Green: $p(f(\mathbf{x}_*), \mathbf{y})$ – Joint Gaussian distribution.

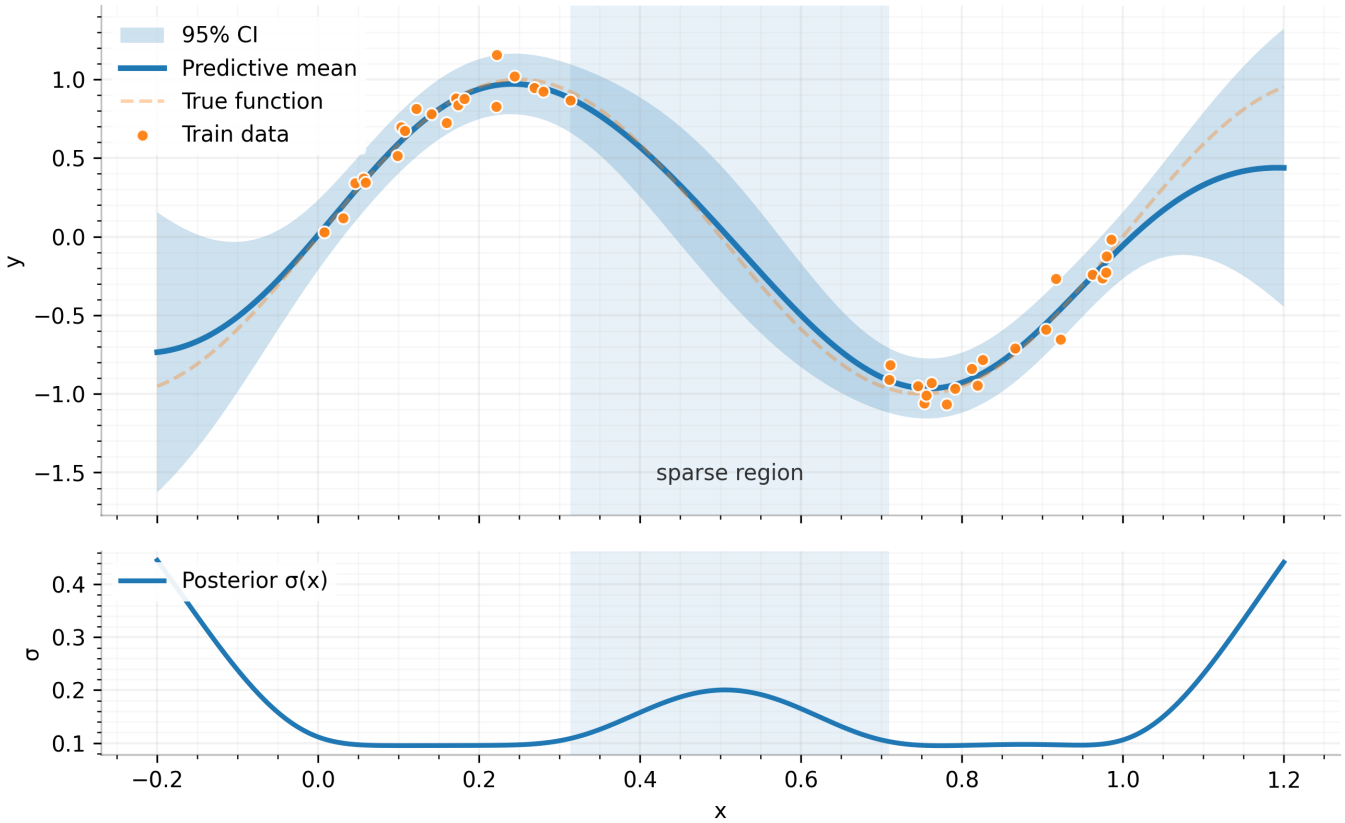
Final Result: The conditional distribution:

$$p(f(\mathbf{x}_*) \mid \mathbf{x}_*, X, \mathbf{y}) = \mathcal{N}(\mu_*, \sigma_*^2).$$

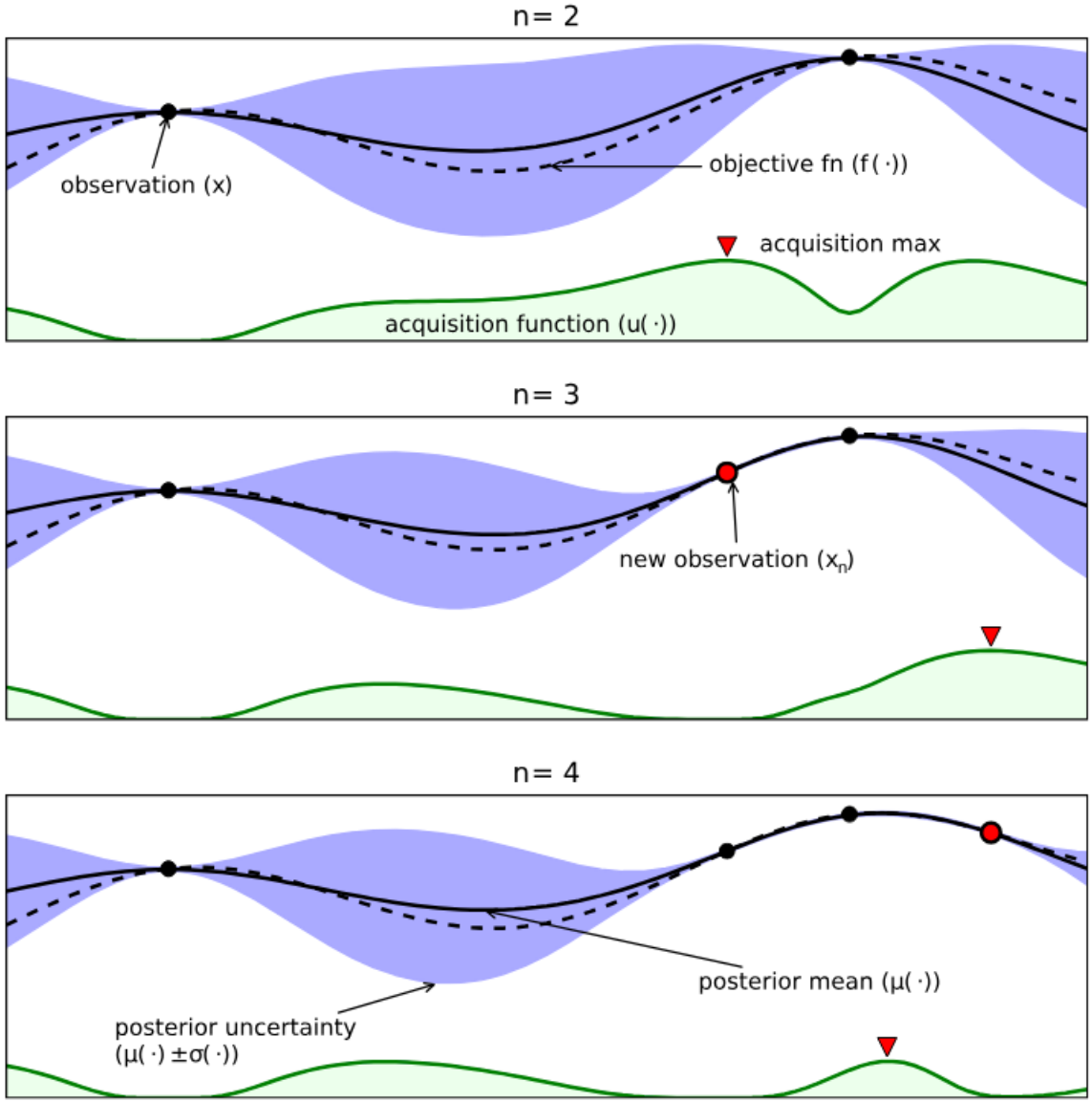
[1] https://en.wikipedia.org/wiki/Multivariate_normal_distribution

Summary of Gaussian Process Regression





The Acquisition Function



[2]

[2] Shahriari, B., et al. (2016). Taking the Human Out of the Loop: A Review of Bayesian Optimization. Proceedings of the IEEE, 104(1), 148–175.

Using Uncertainty to Guide Active Learning

An **acquisition function** $\alpha(\mathbf{x}|\mathcal{D})$ scores how useful it is to evaluate the unknown function f at $\mathbf{x}$ given the current data $\mathcal{D}$.

Optimization (exploitation): maximize the mean prediction.

$$\alpha_{\text{Mean}}(\mathbf{x}) = \max(\mu_n(\mathbf{x})).$$

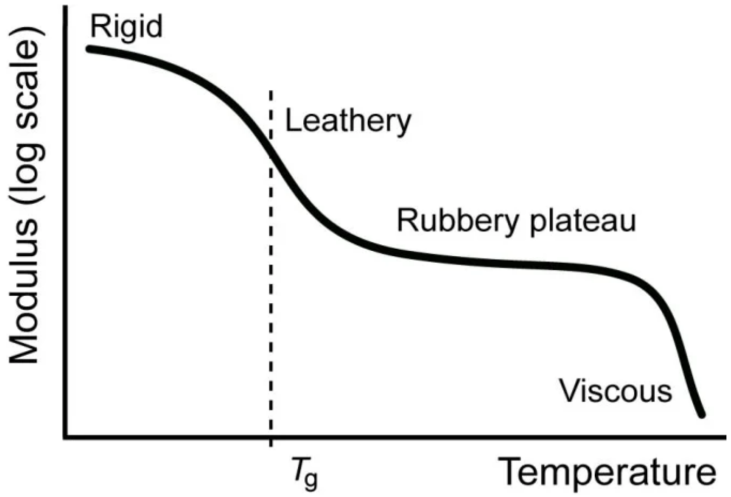
Information gain (exploration): minimize total uncertainty.

$$\alpha_{\text{IG}}(\mathbf{x}) = \underbrace{H[p(f \mid \mathcal{D})]}_{\text{Prior Entropy}} - \underbrace{\mathbb{E}_{y \sim p(y|\mathbf{x},\mathcal{D})}[H[p(f \mid \mathcal{D} \cup \{(\mathbf{x}, y)\})]]}_{\text{Expected Posterior Entropy}}$$

Core trade-off in active learning: *learning the function* vs. *finding its optimum.*

Example: "Active-learning and materials design: the example of high glass transition temperature polymers"

- **Goal:** Find polymers with high glass transition temperature (T_g). [3]
- Glassy Temperature (T_g) is the temperature at which a polymer transitions from a hard, glassy state to a soft, rubbery state.
- Important for plastic materials used at high temperature.



[3] Kim, C., Chandrasekaran, A., Jha, A., & Ramprasad, R. (2019). Active-learning and materials design: The example of high glass transition temperature polymers. MRS Communications, 9(3), 860–866.

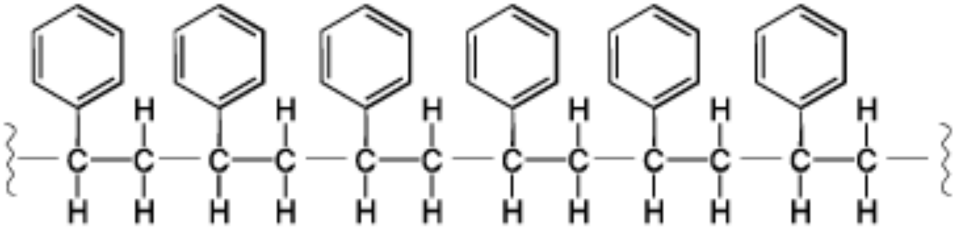
Dataset

- **Dataset:** Online repository with $N = 736$ polymers.
- **Feature set:** $D = 244$ dimensional hierarchical polymer fingerprints.

  Atomic-scale descriptors (123 features): Occurrences of sets of 3 bonded atoms.

  Molecular descriptors (99 features): Quantitative structure-property descriptors, e.g. van der Waals surface area and fraction of rotatable bonds.

  Morphological descriptors (22 features): Larger-scale structural features, e.g. ring spacing and maximum side-chain length.

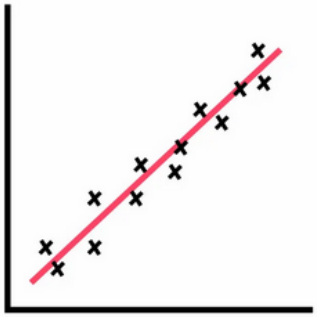


[4]

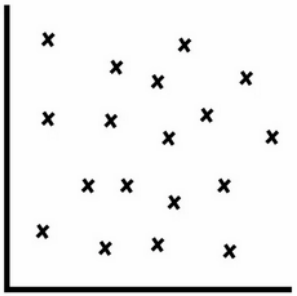
[4] https://letstalkscience.ca/educational-resources/stem-explained/polystyrene-pros-cons-chemistry

Dataset Preprocessing

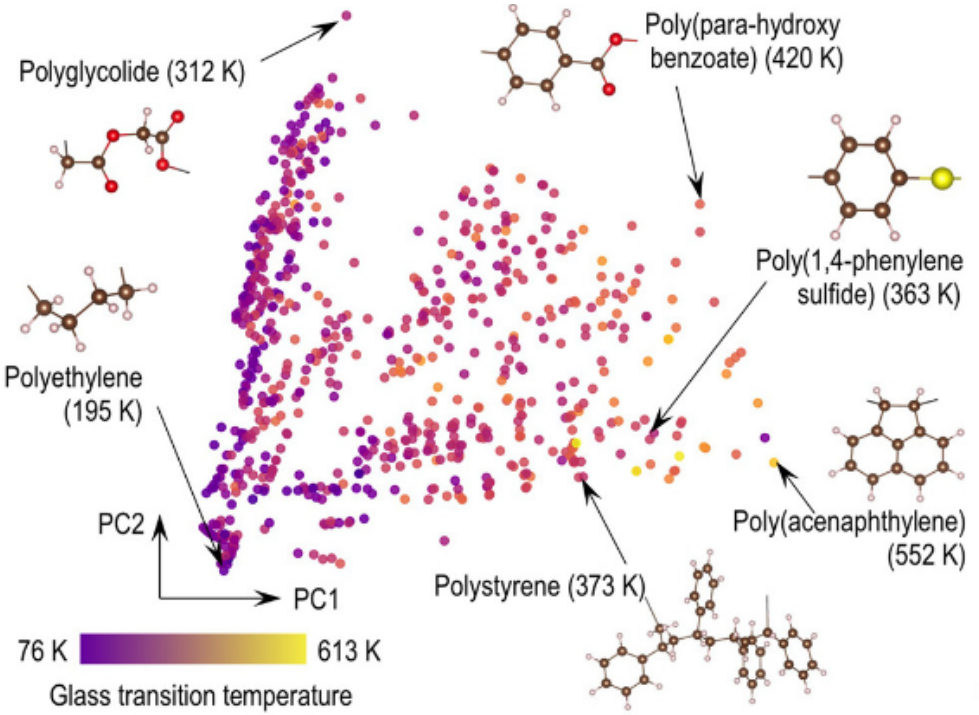
- $D = 244 \rightarrow 2$ with principal component analysis (PCA).
- PCA: Uses correlation to reduce redundant dimensionality.



Positive Correlation



No Correlation

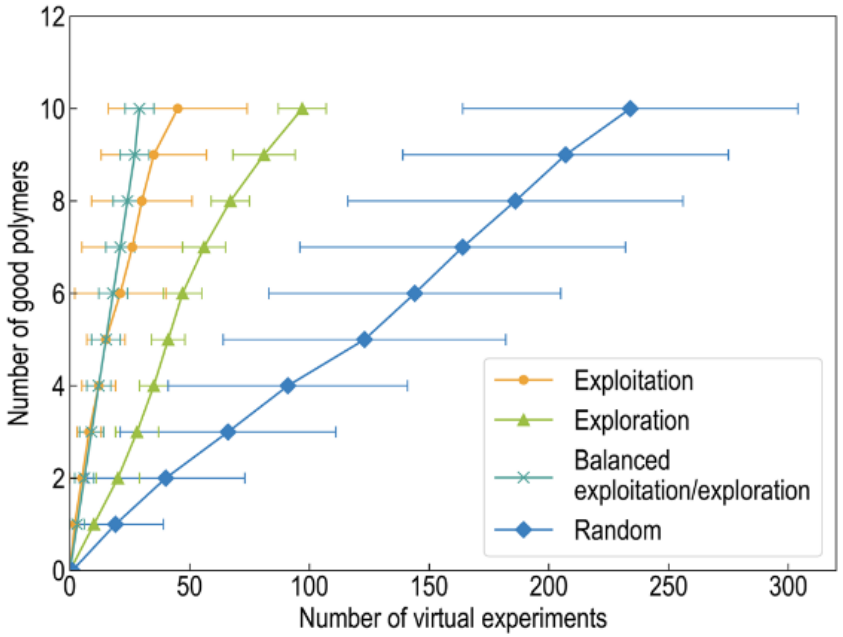


5

[5] Figure from: Kim, C., Chandrasekaran, A., Jha, A., & Ramprasad, R. (2019). Active-learning and materials design: The example of high glass transition temperature polymers. MRS Communications, 9(3), 860–866.

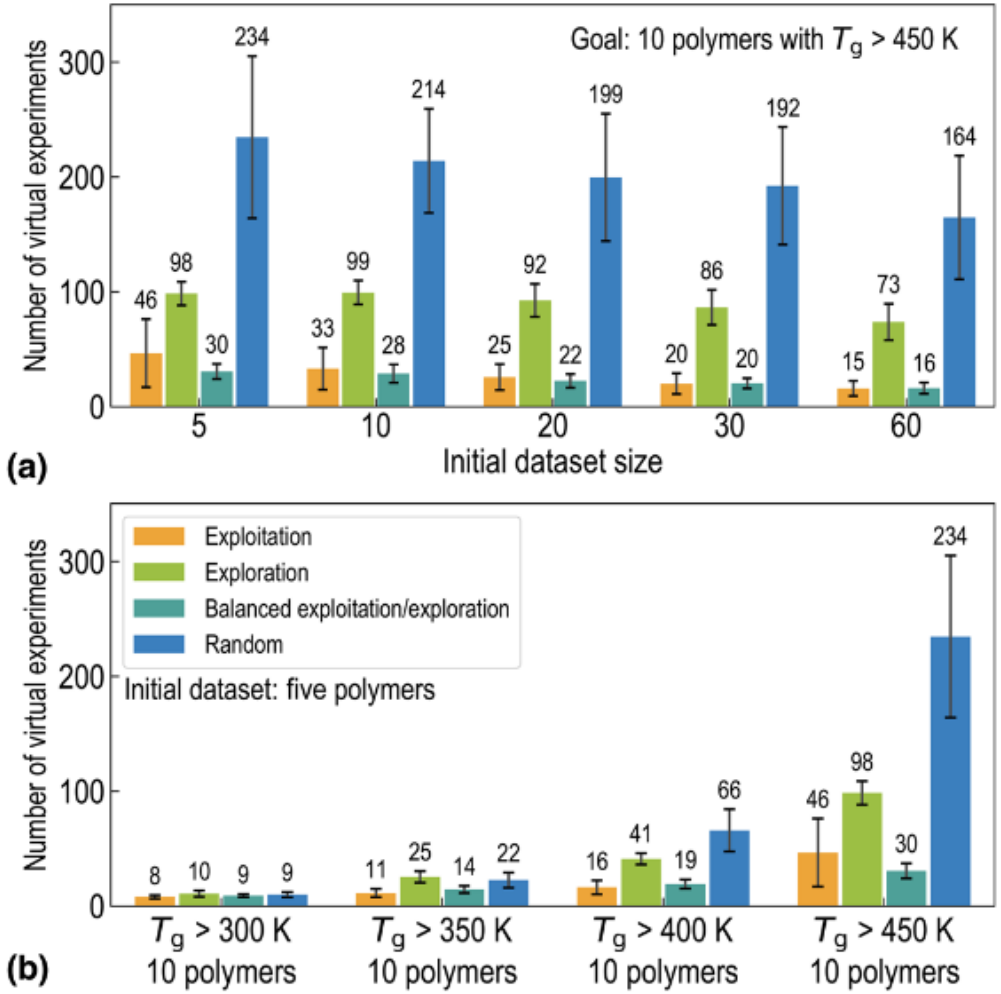
Balanced Exploration–Exploitation Finds More High Tg Polymers

- 4 Strategies compared to find $T_g > 450$ K.
- Best overall strategy: balanced exploration–exploitation.
- Achieves both **high mean performance** and **low variance**.
- Pure exploitation can perform well but is **sensitive to the initial dataset** and can get stuck in local optima.



Balanced Strategies Discover Good Polymers Faster

- Random search is the **least efficient** strategy.
- Exploitation performs well when the **initial dataset is large**.
- Balanced strategies perform best when the **search problem is difficult**.



Conclusion

- Gaussian Processes are a powerful tool for modeling unknown functions with uncertainty.
- Active Learning leverages this uncertainty to strategically select data points that improve the model.
- There is a rich ecosystem of acquisition functions that balance exploration and exploitation.
- We explored an example where acquisition functions were used for optimization but this framework can be applied to many other problems, e.g. learning a function globally.

Thank you

Thank you for your attention!

And an extra thank you to Dr. Kreplak and the 207 office group for their help and feedback on this presentation.

A Gaussian Process is a Distribution over Functions

A **multivariate Gaussian** describes a distribution over vectors:

$$\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}) \implies p(\mathbf{x}) = \frac{1}{(2\pi)^{n/2}|\boldsymbol{\Sigma}|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right).$$

Depends on: the mean $\boldsymbol{\mu}$ and covariance matrix $\boldsymbol{\Sigma}$.

A **Gaussian Process ($\mathcal{GP}$)** is the *infinite-dimensional* generalization of this idea. It describes a distribution over functions $f(x)$:

$$f(x) \sim \mathcal{GP}(m(x), k(x, x')).$$

Depends on: the mean function $m(x)$ and covariance function $k(x, x')$.

Key Idea: Sampling $\mathcal{GP}$ at $\{x_1, x_2, \ldots, x_n\}$ gives a multivariate Gaussian distribution:

$$(f(x_1), f(x_2), \ldots, f(x_n)) \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}).$$

Bayesian Linear Regression (Weight Space)

Model:

$$f(\mathbf{x}) = \mathbf{x}^\top \mathbf{w}$$

Data model (Gaussian noise):

$$y_i = \mathbf{x}_i^\top \mathbf{w} + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma_n^2)$$

For a dataset with design matrix $X \in \mathbb{R}^{n \times d}$:
Likelihood:

$$p(\mathbf{y} \mid X, \mathbf{w}) = \prod_{i=1}^{n} \mathcal{N}(y_i \mid \mathbf{x}_i^\top \mathbf{w}, \sigma_n^2) = \mathcal{N}(X\mathbf{w}, \sigma_n^2 I)$$

Prior on weights:

$$\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \sigma_w^2 I)$$

Bayes' theorem gives a Gaussian posterior:

$$p(\mathbf{w} \mid X, \mathbf{y}) \propto p(\mathbf{y} \mid X, \mathbf{w}) p(\mathbf{w}) = \mathcal{N}(\boldsymbol{\mu}_w, \boldsymbol{\Sigma}_w)$$

Prediction by Marginalizing the Weights

Given a new input $\mathbf{x}_*$, we predict the latent output

$$f_* = \mathbf{x}_*^\top \mathbf{w}.$$

Because the weights are uncertain, we marginalize over their posterior:

$$p(f_* \mid \mathbf{x}_*, X, \mathbf{y}) = \int p(f_* \mid \mathbf{x}_*, \mathbf{w})\, p(\mathbf{w} \mid X, \mathbf{y})\, d\mathbf{w}.$$

Predictive distribution:

$$p(f_* \mid \mathbf{x}_*, X, \mathbf{y}) = \mathcal{N}(\mu_*, \sigma_*^2).$$

Why Gaussian?

The posterior $p(\mathbf{w} \mid X, \mathbf{y})$ is Gaussian.

The prediction $f_* = \mathbf{x}_*^\top \mathbf{w}$ is a linear function of $\mathbf{w}$.

A linear transformation of a Gaussian variable is Gaussian.

Interpretation: Bayesian regression places uncertainty on the weights and propagates this uncertainty to predictions.

Closed-Form Posterior and Predictive

Recall:

$$p(\mathbf{w}) = \mathcal{N}(\mathbf{0}, \sigma_w^2 I), \qquad p(\mathbf{y} \mid X, \mathbf{w}) = \mathcal{N}(X\mathbf{w}, \sigma_n^2 I).$$

Posterior over weights:

$$\boldsymbol{\Sigma}_w = \left(\frac{1}{\sigma_n^2} X^\top X + \frac{1}{\sigma_w^2} I \right)^{-1}$$

$$\boldsymbol{\mu}_w = \frac{1}{\sigma_n^2} \boldsymbol{\Sigma}_w X^\top \mathbf{y}$$

Predictive distribution at $\mathbf{x}_*$:

$$\mu_* = \mathbf{x}_*^\top \boldsymbol{\mu}_w$$

$$\sigma_*^2 = \mathbf{x}_*^\top \boldsymbol{\Sigma}_w \mathbf{x}_* + \sigma_n^2$$

Interpretation:

Posterior mean = regularized least squares solution.

Predictive variance = model uncertainty + observation noise.

Choosing a Good Basis for Infinite Feature Spaces

A naive polynomial feature map

$$\boldsymbol{\phi}(x) = (1, x, x^2, \ldots)^\top$$

induces the kernel

$$k(x, x') = \sum_{n=0}^{\infty} x^n x'^n,$$

This is problematic because:

The kernel is *not stationary* (depends on absolute position).

The basis of the feature space is *not orthogonal*.

The series *does not converge* for $|xx'| \geq 1$.

Using an orthogonal basis with a decaying envelope, e.g.

$$\phi_n(x) = H_n(x)\, e^{-x^2/2},$$

where $H_n(x)$ are Hermite polynomials. Then Σ_w is diagonal with decaying entries, ensuring convergence of the kernel expansion.

Projections into Feature Space (Theory)

Our previous model is limited to linear functions, but we can extend it to non-linear functions by projecting our inputs into a higher-dimensional *feature space* using a mapping $\boldsymbol{\phi}(\mathbf{x})$. For example,

$$\boldsymbol{\phi}(\mathbf{x}) = (1, \mathbf{x}, \mathbf{x}^2, \ldots)^\top.$$

Then our model becomes $f(\mathbf{x}) = \boldsymbol{\phi}(\mathbf{x})^\top \mathbf{w}$. Since everything is still linear in $\mathbf{w}$, all of our previous results still hold, we just need to replace $\mathbf{x}$ with $\boldsymbol{\phi}(\mathbf{x})$ everywhere so the predictive distribution is now normally distributed: $\mathcal{N}\left(\mu_*, \sigma_*^2\right)$.

$$\mu_* = \boldsymbol{\phi}(\mathbf{x}_*)^\top \Sigma_w \boldsymbol{\Phi} \left(\boldsymbol{\Phi}^\top \Sigma_w \boldsymbol{\Phi} + \sigma_n^2 \mathbf{I}\right)^{-1} \mathbf{y},$$

$$\sigma_*^2 = \boldsymbol{\phi}(\mathbf{x}_*)^\top \Sigma_w \boldsymbol{\phi}(\mathbf{x}_*) - \boldsymbol{\phi}(\mathbf{x}_*)^\top \Sigma_w \boldsymbol{\Phi} \left(\boldsymbol{\Phi}^\top \Sigma_w \boldsymbol{\Phi} + \sigma_n^2 \mathbf{I}\right)^{-1} \boldsymbol{\Phi}^\top \Sigma_w \boldsymbol{\phi}(\mathbf{x}_*).$$

Then we define the kernel function as $k(\mathbf{x}, \mathbf{x}') = \boldsymbol{\phi}(\mathbf{x})^\top \Sigma_w \boldsymbol{\phi}(\mathbf{x}')$, which allows us to rewrite the predictive mean and variance as

$$\mu_* = \mathbf{k}_*^\top \left(K + \sigma_n^2 \mathbf{I}\right)^{-1} \mathbf{y}, \quad \sigma_*^2 = k(\mathbf{x}_*, \mathbf{x}_*) - \mathbf{k}_*^\top \left(K + \sigma_n^2 \mathbf{I}\right)^{-1} \mathbf{k}_*,$$

$$\text{where } K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j) \quad \mathbf{k}_* = \left[k(\mathbf{x}_*, \mathbf{x}_1), \ldots, k(\mathbf{x}_*, \mathbf{x}_n)\right]^\top$$

Gaussian Process Regression – Math

Given the dataset $X = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$, $\mathbf{y} = (y_1, \ldots, y_n)$, we have a $\mathcal{GP}$ prior

$$\mathbf{y} \sim \mathcal{N}(\mathbf{0}, K(X, X)), \quad \text{where} \quad K(X, X) = \begin{pmatrix} k(\mathbf{x}_1, \mathbf{x}_1) & \cdots & k(\mathbf{x}_1, \mathbf{x}_n) \\ \vdots & \ddots & \vdots \\ k(\mathbf{x}_n, \mathbf{x}_1) & \cdots & k(\mathbf{x}_n, \mathbf{x}_n) \end{pmatrix}$$

Sampling a $\mathcal{GP}$ at X and a new point $\mathbf{x}_*$ gives a **joint Gaussian**:

$$\begin{bmatrix} \mathbf{y} \\ f(\mathbf{x}_*) \end{bmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{bmatrix} K(X, X) & K(X, \mathbf{x}_*) \\ K(\mathbf{x}_*, X) & K(\mathbf{x}_*, \mathbf{x}_*) \end{bmatrix}\right).$$

Prediction are obtained by conditioning the joint Gaussian:

$$p(f(\mathbf{x}_*) \mid \mathbf{x}_*, X, \mathbf{y}) = \mathcal{N}(\mu_*, \sigma_*^2).$$

where μ_* and σ_*^2 are given by closed-form expressions in terms of the kernel.

Acquisition Function – Expected Improvement (Theory)

Expected Improvement (EI): for the GP predictive distribution $f(\mathbf{x}) \sim \mathcal{N}(\mu_n(\mathbf{x}), \sigma_n^2(\mathbf{x}))$ relative to a threshold τ (e.g. the best observed value). [6] Define the improvement random variable

$$I(\mathbf{x}) = \max(0, f(\mathbf{x}) - \tau).$$

The expected improvement is

$$\begin{aligned}
\alpha_{\mathrm{EI}}(\mathbf{x}) &= \mathbb{E}[I(\mathbf{x})] \\
&= \int_{\tau}^{\infty} (f - \tau)\mathcal{N}(f \mid \mu_n(\mathbf{x}), \sigma_n^2(\mathbf{x}))df \\
&= \int_{\tau}^{\infty} (f - \tau)\frac{1}{\sqrt{2\pi}\sigma_n(\mathbf{x})} \exp\left(-\frac{(f - \mu_n(\mathbf{x}))^2}{2\sigma_n^2(\mathbf{x})}\right) df \\
&= (\mu_n(\mathbf{x}) - \tau)\Phi(z) + \sigma_n(\mathbf{x})\phi(z), \quad z = \frac{\mu_n(\mathbf{x}) - \tau}{\sigma_n(\mathbf{x})}
\end{aligned}$$

and Φ and ϕ are the CDF and PDF of the standard normal distribution. The LHS term is large when the mean is much better than the threshold, and the RHS term is large when there is high uncertainty.

[6] Shahriari et al. (2016). Taking the Human Out of the Loop: A Review of Bayesian Optimization.

Acquisition Function – Information Gain (Theory)

Information Gain (IG): how much we expect our uncertainty about f to decrease by sampling at $\mathbf{x}$:

$$\alpha_{\mathrm{IG}}(\mathbf{x}) = H[p(f \mid \mathcal{D})] - \mathbb{E}_{y \sim p(y \mid \mathbf{x}, \mathcal{D})}[H[p(f \mid \mathcal{D} \cup \{(\mathbf{x}, y)\}]] \,.$$

Prior Entropy:

$$H[p(f \mid \mathcal{D})] = -\int p(f \mid \mathcal{D}) \log p(f \mid \mathcal{D}) \, df$$

$$= -\int \mathcal{N}(f \mid \mu_n(\mathbf{x}), \sigma_n^2(\mathbf{x})) \log \mathcal{N}(f \mid \mu_n(\mathbf{x}), \sigma_n^2(\mathbf{x})) \, df = \frac{1}{2} \log\big(2\pi e \, \sigma_n^2(\mathbf{x})\big).$$

Posterior Entropy: after observing a new point, the entropy has the same form but with the updated posterior variance:

$$H[p(f \mid \mathcal{D} \cup \{(\mathbf{x}, y)\})] = \frac{1}{2} \log\big(2\pi e \, \sigma_{n+1}^2(\mathbf{x})\big).$$

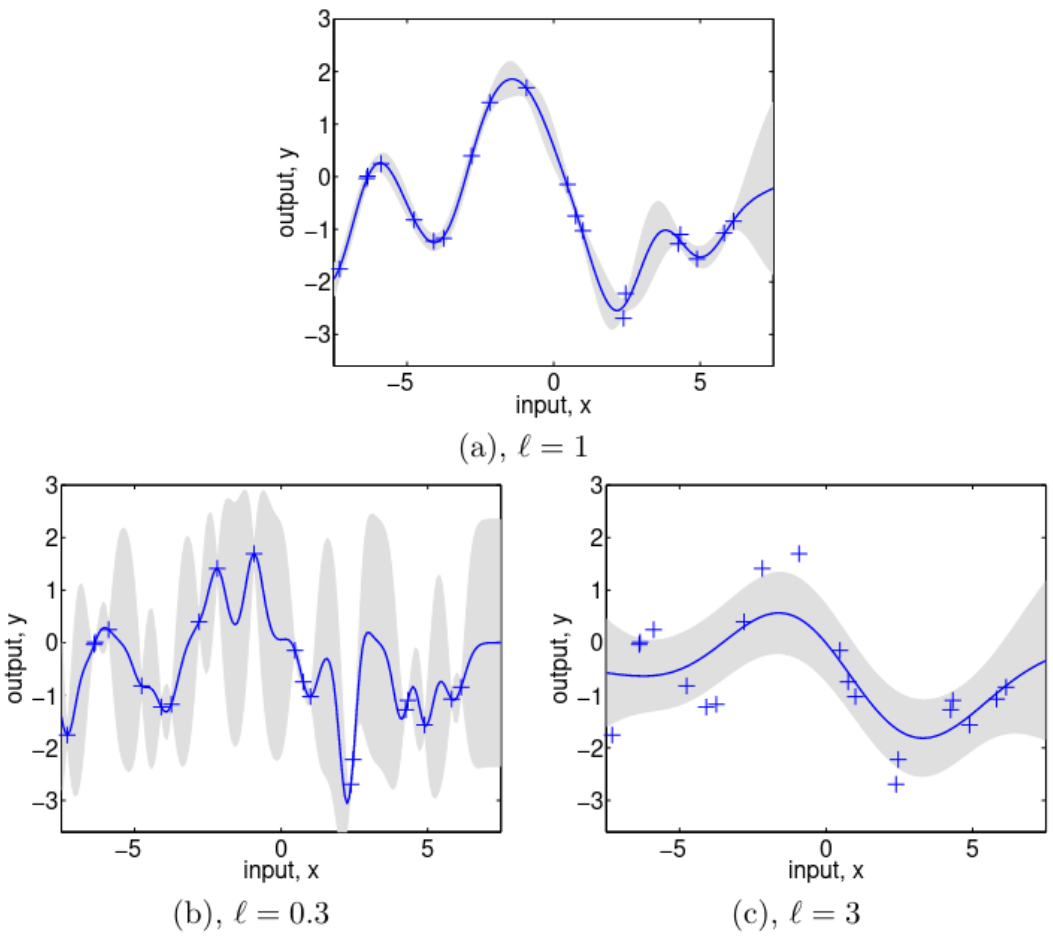
Therefore,

$$\alpha_{\mathrm{IG}}(\mathbf{x}) = \frac{1}{2} \log\big(2\pi e \, \sigma_n^2(\mathbf{x})\big) - \frac{1}{2} \log\big(2\pi e \, \sigma_{n+1}^2(\mathbf{x})\big) = \frac{1}{2} \log\left(\frac{\sigma_n^2(\mathbf{x})}{\sigma_{n+1}^2(\mathbf{x})} \right).$$

Gaussian Process Regression (Hyperparameters)

- The standard kernel for GPs is the RBF (squared exponential) kernel:

$$k(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\ell^2}\right),$$



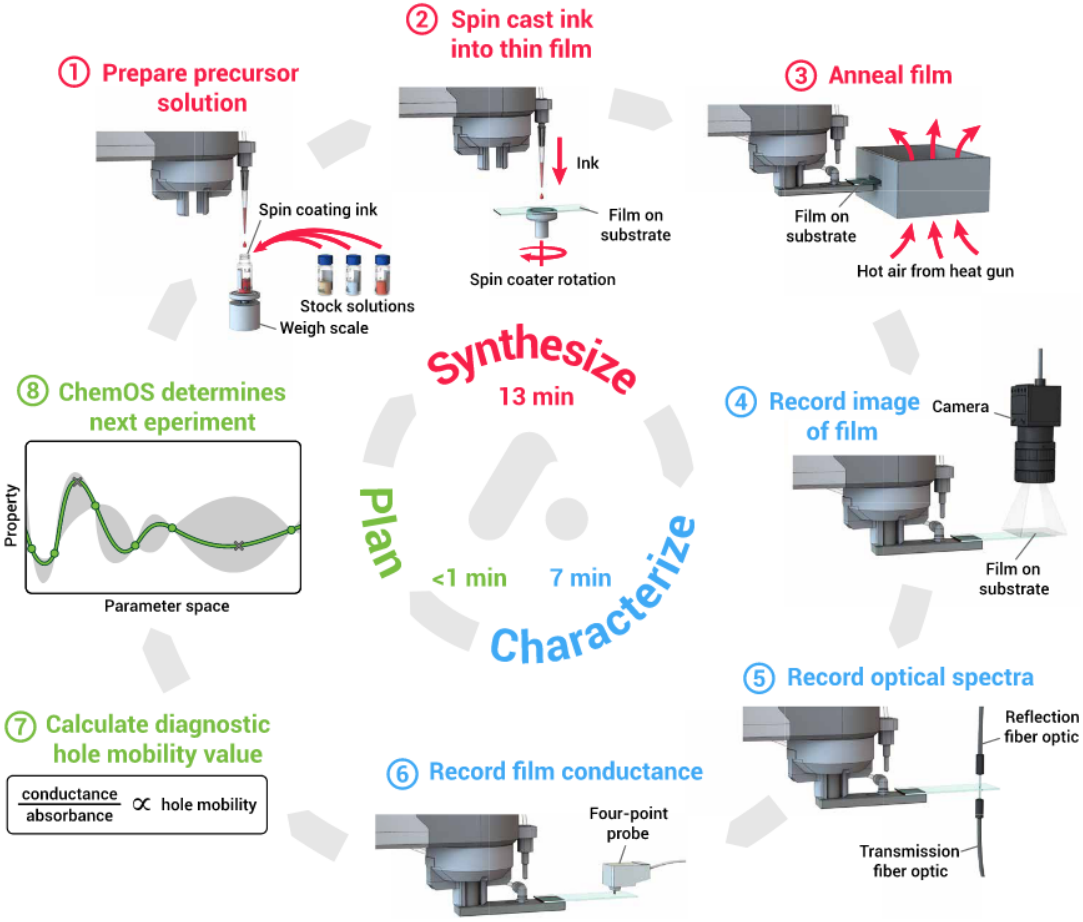
(a), $\ell = 1$

(b), $\ell = 0.3$

(c), $\ell = 3$

Example: Thin Film Pseudomobility with Automated Experimentation

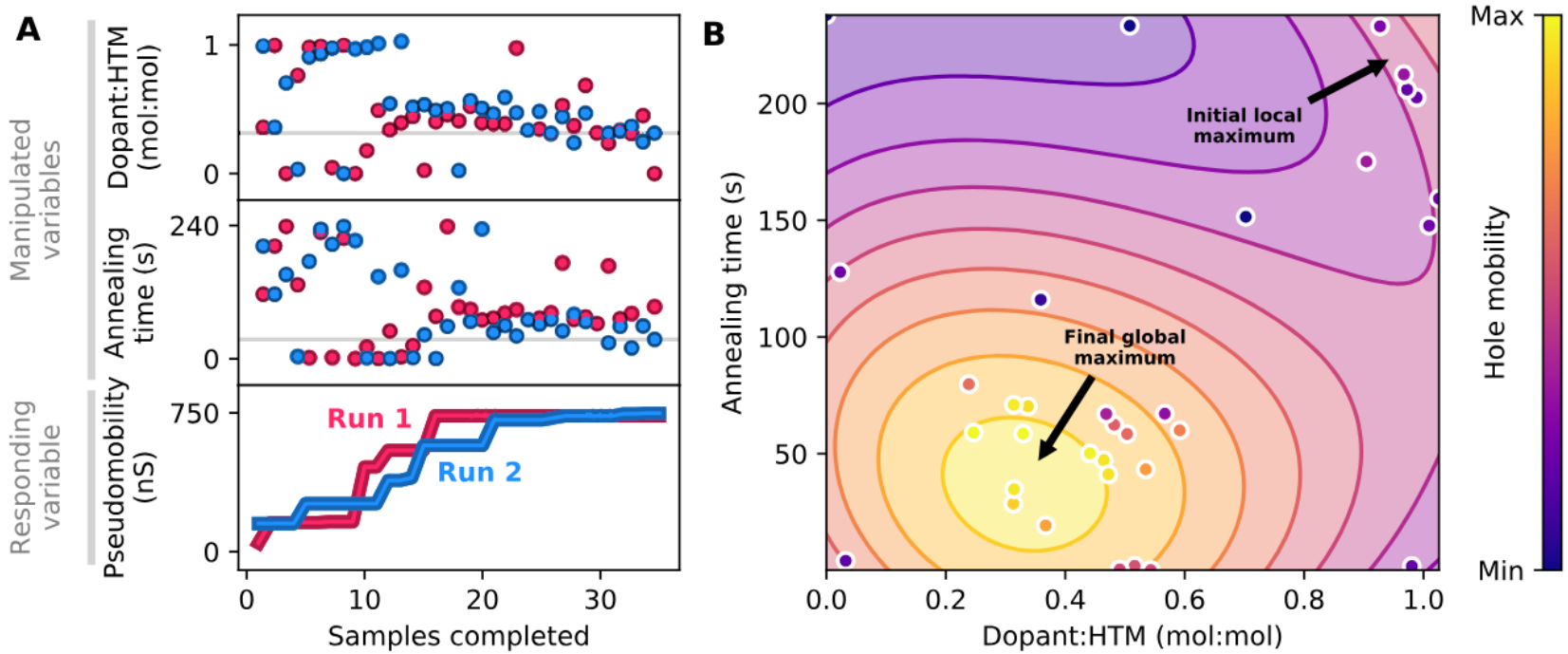
- Example of using active learning to optimize the pseudomobility of a thin film. [7]



[7] MacLeod, B. P., et al. (2020). Self-driving laboratory for accelerated discovery of thin-film materials. Science Advances, 6(20), eaaz8867.

Example: Thin Film Pseudomobility (Results)

- This just looked for the minimum of a 2D function.
- Example of the strategy called **Expected Improvement**.
- Parameters: Dopant concentration and annealing time.
- Output: Pseudomobility of the thin film.



Hierarchical Polymer Fingerprints: Atomic Descriptors

Local chemical environments are captured using **atomic triples**.

An atomic triple is a sequence of three bonded atoms.

Example: $O{-}C{-}C$

Each atom is labeled by its **coordination number** (number of bonds).

$$C_4{-}C_3{-}O_2$$

The fingerprint records how often each triple occurs in the polymer repeat unit.

For this dataset there are **123 possible triples**.

Purpose: capture the **local bonding chemistry** that influences stiffness and chain mobility.

Hierarchical Polymer Fingerprints: Molecular Descriptors

Quantitative structure-property descriptors are computed from the molecular structure.

Examples include:

van der Waals surface area

Topological surface area

Fraction of rotatable bonds

These descriptors capture global molecular properties that affect polymer flexibility and packing.

All descriptors are computed using the **RDKit cheminformatics library**.

This level contributes **99 features** to the fingerprint.

Purpose: capture physical properties that influence chain mobility and intermolecular interactions.

Hierarchical Polymer Fingerprints: Morphological Descriptors

The third level captures **larger-scale structural features** of the polymer.

Examples include:

- Topological distance between rings
- Fraction of atoms in side chains
- Length of the largest side chain

These descriptors characterize the **overall architecture** of the polymer chain.

Structural morphology strongly influences **packing, rigidity, and glass transition temperature**.

This level contributes **22 morphological descriptors**.

Purpose: capture how the polymer structure extends beyond local chemical bonds.

Infinite Feature Maps are Not a Problem

Recall: $\mathcal{GP}$ given by **mean** and **covariance** function.

The **mean function** is[8]:

$$m(x) = \mathbb{E}[f(x)] = \boldsymbol{\phi}(x)^\top \mathbb{E}[\mathbf{w}] = 0,$$

The **covariance function** contains *all of the information*:

$$k(x, x') = \mathbb{E}\big[f(x)f(x')\big] = \boldsymbol{\phi}(x)^\top \mathbb{E}[\mathbf{w}\mathbf{w}^\top]\boldsymbol{\phi}(x') = \sigma_w^2 \boldsymbol{\phi}(x)^\top \boldsymbol{\phi}(x').$$

We DO NOT need to compute the weights, but we DO need to compute large inner products.

[8] Prior: $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \sigma_w^2 I)$

The Kernel Trick

Chose a specific feature map (there are other options):

$$\boldsymbol{\phi}(x) = \left(1,\, x,\, \frac{x^2}{\sqrt{2!}},\, \frac{x^3}{\sqrt{3!}},\, \ldots\right).$$

The kernel becomes

$$k(x, x') = \boldsymbol{\phi}(x)^\top \boldsymbol{\phi}(x') = \sum_{n=0}^{\infty} \frac{(xx')^n}{n!}.$$

This is the Taylor expansion of the exponential function:

$$k(x, x') = e^{xx'}.$$

Simple to compute but retains complexity.